

```
import json

import os

from datetime import datetime, timedelta


DB_FILE = "database.txt"

DB_PATH = DB_FILE


# Состояние приложения, общее для всех модулей

app_state = {

    "active_tab": 0,

    "newly_added_cars": [],

    "view_mode": "list",

    "selected_car": None

}


def get_default_car_data():

    """Генерирует демонстрационный шаблон данных для первого запуска."""

    current_date = datetime.now().strftime("%d.%m.%Y")

    past_date = (datetime.now() - timedelta(days=30)).strftime("%d.%m.%Y")

    return {

        "odometer": {"value": 125000, "date": current_date},

        "daily_mileage": 45,

        "odometer_history": [

            {"value": 123650, "date": past_date},

            {"value": 125000, "date": current_date},

        ],

        "maintenance_data": {

            "Замена масла + фильтры": {"last_service": 120000, "interval": 10000, "date": current_date},

            "Замена ГРМ (ремень, помпа)": {"last_service": 90000, "interval": 60000, "date": current_date},
```

```

        "Замена антифриза": {"last_service": 100000, "interval": 50000, "date": current_date},

        "Тормозная жидкость": {"last_service": 100000, "interval": 40000, "date": current_date},

        "Обслуживание кондиционера": {"last_service": 110000, "interval": 30000, "date":
current_date},

    },

    "history": [],

}

```

```

def parse_h_date(item):

```

```

    """Глобальная функция парсинга дат для сортировки истории."""

```

```

    try:

```

```

        return datetime.strptime(item["date"], "%d.%m.%Y")

```

```

    except:

```

```

        return datetime.min

```

```

def recalculate_auto_daily_mileage(car_profile):

```

```

    """Счет реального пробега в сутки по истории."""

```

```

    history = car_profile.get("odometer_history", [])

```

```

    if len(history) < 2:

```

```

        return int(car_profile.get("daily_mileage", 45))

```

```

    try:

```

```

        sorted_hist = sorted(history, key=lambda x: datetime.strptime(x["date"], "%d.%m.%Y"))

```

```

        first, last = sorted_hist[0], sorted_hist[-1]

```

```

        d1 = datetime.strptime(first["date"], "%d.%m.%Y")

```

```

        d2 = datetime.strptime(last["date"], "%d.%m.%Y")

```

```

        days = (d2 - d1).days

```

```

        km = last["value"] - first["value"]

```

```

        if days > 0 and km > 0:

```

```

            return max(1, int(km / days))

```

except Exception:

pass

return int(car\_profile.get("daily\_mileage", 45))

def load\_data():

"""Загружает базу из файла и адаптирует под обновления."""

if not os.path.exists(DB\_FILE):

initial\_data = {"cars": {"Мой Автомобиль": get\_default\_car\_data()}}

save\_data(initial\_data)

return initial\_data

try:

with open(DB\_FILE, "r", encoding="utf-8") as f:

data = json.load(f)

if "cars" not in data:

data = {"cars": {}}

for car\_name, car\_profile in data["cars"].items():

if car\_name in app\_state["newly\_added\_cars"] and car\_profile.get("odometer", {}).get("value")  
== 0:

if "odometer" not in car\_profile:

car\_profile["odometer"] = {"value": 0, "date": datetime.now().strftime("%d.%m.%Y")}

if "daily\_mileage" not in car\_profile:

car\_profile["daily\_mileage"] = 0

if "odometer\_history" not in car\_profile:

car\_profile["odometer\_history"] = []

if "maintenance\_data" not in car\_profile:

car\_profile["maintenance\_data"] = {}

if "history" not in car\_profile:

car\_profile["history"] = []

continue

if "odometer" not in car\_profile:

car\_profile["odometer"] = {"value": 125000, "date": datetime.now().strftime("%d.%m.%Y")}

if "daily\_mileage" not in car\_profile:

car\_profile["daily\_mileage"] = 45

if "odometer\_history" not in car\_profile:

car\_profile["odometer\_history"] = []

if "maintenance\_data" not in car\_profile:

car\_profile["maintenance\_data"] = {}

if "history" not in car\_profile:

car\_profile["history"] = []

car\_profile["daily\_mileage"] = recalculate\_auto\_daily\_mileage(car\_profile)

for task\_name, task\_info in car\_profile["maintenance\_data"].items():

if "last\_service" not in task\_info:

task\_info["last\_service"] = car\_profile["odometer"]["value"]

if "interval" not in task\_info:

task\_info["interval"] = 10000

if "date" not in task\_info:

task\_info["date"] = datetime.now().strftime("%d.%m.%Y")

return data

except Exception:

return {"cars": {}}

def save\_data(data):

"""Преобразует словарь данных в JSON и пишет на диск."""

try:

```
with open(DB_FILE, "w", encoding="utf-8") as f:
    json.dump(data, f, ensure_ascii=False, indent=4)
except Exception as e:
    print(f"Ошибка записи на диск: {e}")
```